

中山大学计算机学院

编译器构造实验 Lab3 实验二报告

JFlex 自动生成 Oberon-0 词法分析程序

本报告对应 ROSE 项目实验二：总结词汇表、抽取词法规则正则定义式、编写 JFlex 输入源文件、实现词法扫描器、比较不同 lex 族工具差异。

课程	编译器构造实验 (DCS292)
实验名称	Lab3 实验二：JFlex 词法分析
姓名	梁力航
学号	23336128
日期	2026 年 5 月 20 日
运行环境	OpenJDK 11.0.24 / macOS / JFlex 1.8.2

保留字

19

关键字

7

词法错误

6 种

异常类

16 个

测试通过

6/6

目录

1	词法分析器总体架构	2
2	Oberon-0 词汇表	2
2.1	保留字与关键字分类	2
2.2	运算符优先级与结合性	3
2.3	分隔符与定界符	3
3	词法规则的正则定义式	4
3.1	核心正则定义	4
3.2	关键约束规则	4
4	词法分析核心算法	4
4.1	整数扫描有限状态自动机 (DFA)	4
4.2	标识符与保留字识别	6
4.3	注释处理	7
4.4	运算符与分隔符识别	7
5	JFlex 输入源文件 oberon.flex	7
5.1	文件结构	7
5.2	关键规则片段	8
6	异常类层次设计	9
7	Lex 族工具差异比较	11
7.1	三代 Lex 工具对比	11
7.2	关键差异分析	11
8	测试结果	11
8.1	正确程序扫描	11
8.2	词法错误检测汇总	12

1 词法分析器总体架构

词法分析器（Scanner）是整个 ROSE 工具链的第一个阶段。它的职责是将 Oberon-0 源程序的字符序列转换为 token 序列，供语法分析器消费。

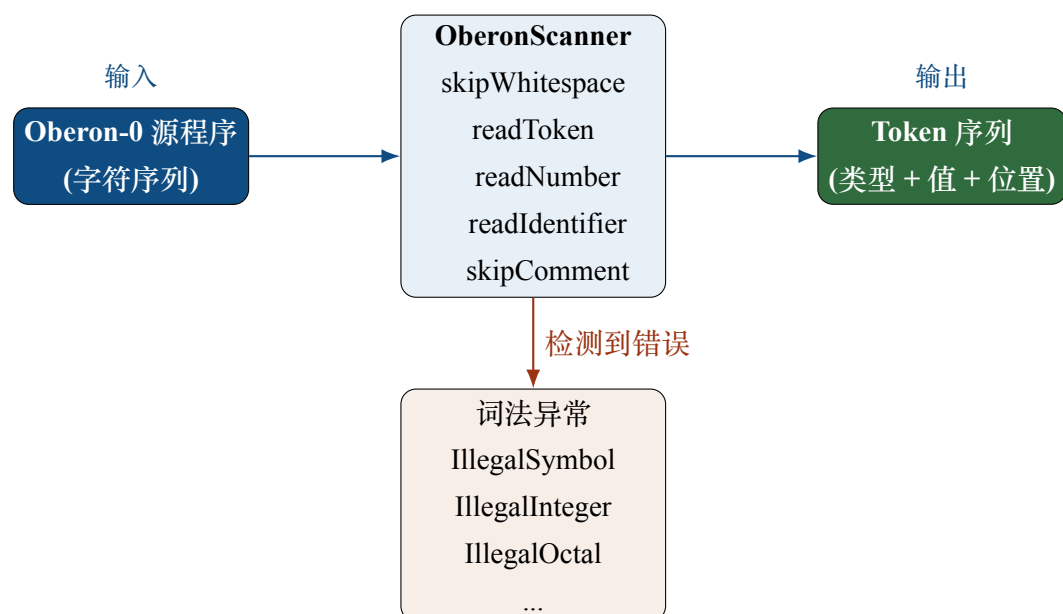


图 1: 词法分析器总体架构——字符流 → Token 流

扫描器采用单字符前瞻（**one-char lookahead**）的迭代模型：每次读取一个字符，根据首字符分派到对应的子自动机（数字 → readNumber、字母 → readIdentifier、运算符 → 直接匹配），跳过空白后在首字符处做出分派决策。

2 Oberon-0 词汇表

2.1 保留字与关键字分类

保留字（19 个）——语法结构的一部分		关键字（7 个）——预定义标识符	
模块结构	MODULE, BEGIN, END	预定义类型	INTEGER, BOOLEAN
声明	CONST, TYPE, VAR, PROCEDURE	布尔值	TRUE, FALSE
控制流	IF, THEN, ELSIF, ELSE, WHILE, DO	预定义过程	READ, WRITE, WRITELN
类型构造	ARRAY, OF, RECORD		
运算符保留	DIV, MOD, OR		

实现中的关键决策：手写 OberonScanner 将 INTEGER、BOOLEAN、TRUE、FALSE 作为普通 IDENTIFIER 返回——因为在 Oberon-0 中它们是关键字（可被覆盖），不是保留字。类型名和布尔值的语义由语法分析阶段根据上下文决定。

2.2 运算符优先级与结合性



图 2: Oberon-0 运算符优先级层次——仅 5 级 + 单目

2.3 分隔符与定界符

;	语句分隔	.	域选择 / 模块结束	,	参数/列表分隔	:	类型标注
:=	赋值	()	分组/参数	[]	数组下标	(* *)	注释

3 词法规则的正则定义式

3.1 核心正则定义

字符类		Token 规则	
letter	[a-zA-Z]	identifier	letter (letter digit)*
digit	[0-9]	DecimalInteger	[1-9] digit*
octal_digit	[0-7]	OctalInteger	0 octal_digit*
WhiteSpace	[\t\f\r\n]	number	DecimalInteger OctalInteger
		comment	"(*" ... "*" + ")"

表 1: 词法规则正则定义式汇总

3.2 关键约束规则

1. 大小写无关: Oberon-0 不区分大小写——MODULE = Module = module。扫描器将所有标识符和保留字归一化为小写后匹配
2. 整数常量与标识符必须空白分隔: 25id 是非法整数 (IllegalIntegerException), 不允许解释为 25 + id
3. 八进制约束: 0 开头的整数为八进制, 仅允许数字 0-7; 018 非法
4. 标识符长度: ≤ 24 字符
5. 整数长度: ≤ 12 位数字 (十进制或八进制)
6. 注释不允许嵌套: (* ... (* ... *) ... *) 中, 第一个 *) 即关闭注释

4 词法分析核心算法

4.1 整数扫描有限状态自动机 (DFA)

Oberon-0 的整数常量分为十进制和八进制, 且必须满足长度和数字范围约束。下面给出数值扫描的完整 DFA:

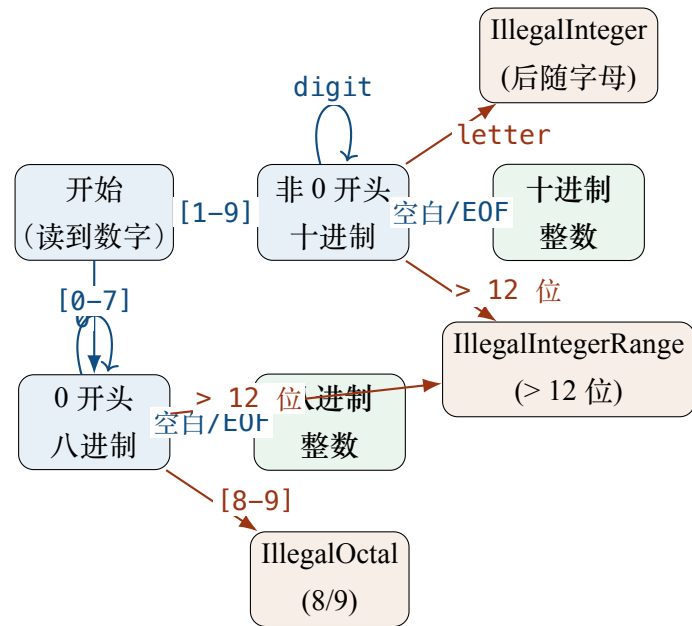


图 3: 整数常量 DFA——十进制/八进制分流 + 三种错误转换

Listing 1: OberonScanner.readNumber() 核心逻辑

```

private Token readNumber(int startLine, int startCol)
    throws IOException, LexicalException {
    StringBuilder sb = new StringBuilder();
    boolean isOctal = (currentChar == '0');

    while (currentChar != -1 && Character.isDigit((char) currentChar)) {
        sb.append((char) currentChar);
        advance();
    }

    String text = sb.toString();
    // Check length: max 12 digits
    if (text.length() > 12)
        throw new IllegalIntegerRangeException(...);
    // Check illegal concatenation with following identifier
    if (currentChar != -1 && Character.isLetter((char) currentChar))
        throw new IllegalIntegerException(...);
    // Check octal validity
    if (isOctal && text.length() > 1) {
        for (int i = 1; i < text.length(); i++)
            if (text.charAt(i) == '8' || text.charAt(i) == '9')
                throw new IllegalOctalException(...);
        return new Token(INTEGER_LITERAL, Integer.parseInt(text, 8), ...);
    }
    return new Token(INTEGER_LITERAL, Integer.parseInt(text, 10), ...);
}

```

}

4.2 标识符与保留字识别

标识符识别采用“先收集、后分类”策略：连续读取字母和数字形成词素，然后查表判断是保留字还是普通标识符。

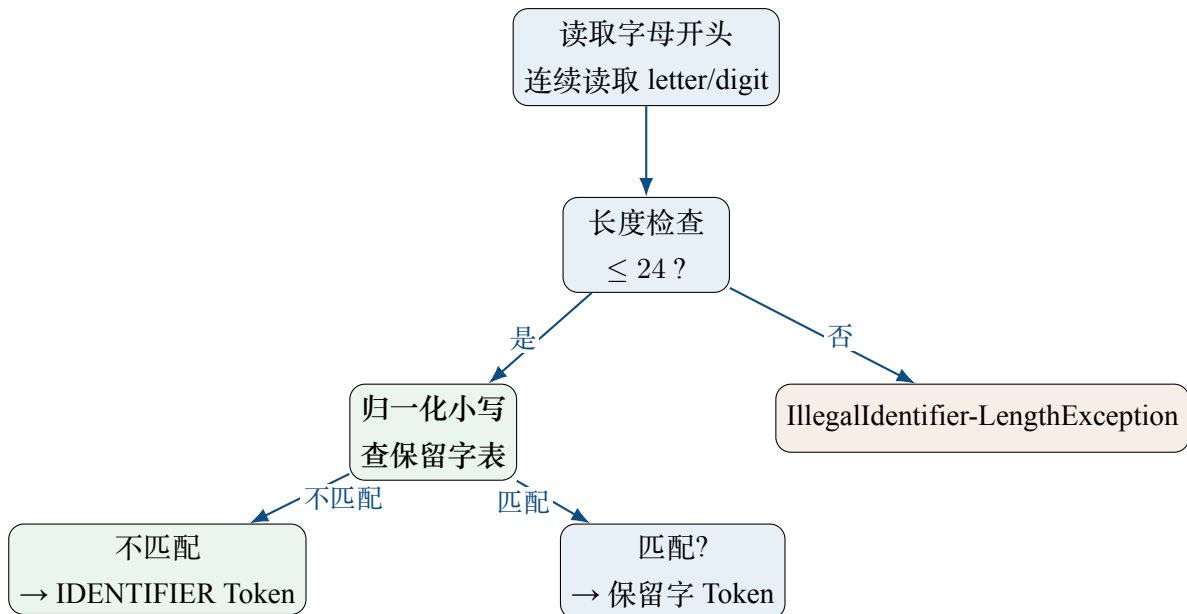


图 4: 标识符/保留字识别流程

Listing 2: 保留字查表（19 个保留字 + 3 个预定义过程）

```

private Token readIdentifierOrReservedWord(...) {
    // ... read letters + digits ...
    String lower = text.toLowerCase();
    switch (lower) {
        case "module":    return new Token(MODULE, lower, ...);
        case "begin":     return new Token(BEGIN, lower, ...);
        // ... 17 more reserved words ...
        case "read":      return new Token(READ, lower, ...);
        case "write":     return new Token(WRITE, lower, ...);
        case "writeln":   return new Token(WRITELN, lower, ...);
        default:          return new Token(IDENTIFIER, lower, ...);
    }
}

```

Oberon-0 的 INTEGER、BOOLEAN、TRUE、FALSE 是关键字（不是保留字），在扫描器中落入 default 分支，以 IDENTIFIER 类型返回。它们作为类型名和布尔值的语义在语法分析阶段按上下文解析。

4.3 注释处理

Oberon-0 注释以 (*) 开始、*) 结束，不允许嵌套。扫描器使用简单的循环匹配：

Listing 3: 注释扫描核心逻辑

```
private void skipComment() throws IOException, MismatchedCommentException {
    int startLine = line, startCol = column - 2;
    while (true) {
        if (currentChar == -1)
            throw new MismatchedCommentException(
                "Unclosed comment starting at line " + startLine);
        if (currentChar == '*') {
            advance();
            if (currentChar == ')') { advance(); return; }
        } else {
            advance();
        }
    }
}
```

4.4 运算符与分隔符识别

双字符运算符 (:=、<=、>=) 通过向前看一个字符区分：

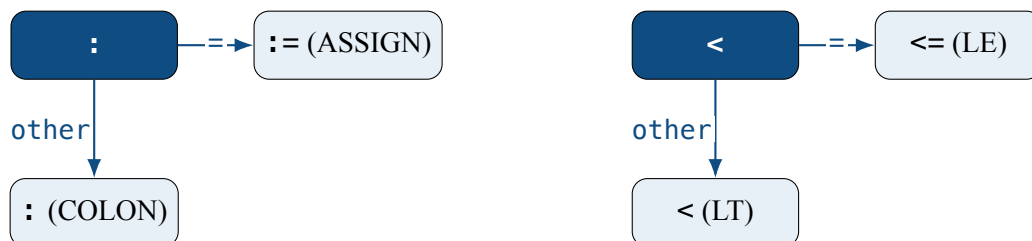


图 5: 双字符运算符的向前看匹配 (:= 和 <=)

5 JFlex 输入源文件 oberon.flex

5.1 文件结构

JFlex 输入文件分为三个段落，由 %% 分隔：

段落	内容	说明
用户代码	<code>import</code> 、辅助方法	复制到生成的 <code>Scanner</code> 类中
<code>%%</code>	分隔符	
选项/声明	<code>%class</code> , <code>%cup</code> , <code>%type</code>	指定类名、CUP 集成、状态宏定义
<code>%%</code>	分隔符	
词法规则	正则 <code>{</code> <code>action }</code>	每个模式对应一个语义动作

5.2 关键规则片段

Listing 4: oberon.flex — 选项段与词法状态

```
%class OberonScanner
%unicode
%line
%column
%type java_cup.runtime.Symbol
%cup
%public
```

Listing 5: oberon.flex — 保留字规则（19 个）

```
"MODULE"    { return symbol(sym.MODULE); }
"BEGIN"     { return symbol(sym.BEGIN); }
"END"       { return symbol(sym.END); }
"CONST"     { return symbol(sym.CONST); }
"TYPE"      { return symbol(sym.TYPE); }
"VAR"       { return symbol(sym.VAR); }
"PROCEDURE" { return symbol(sym.PROCEDURE); }
"IF"        { return symbol(sym.IF); }
// ... THEN, ELIF, ELSE, WHILE, DO, ARRAY, OF, RECORD
"DIV"       { return symbol(sym.DIV); }
"MOD"       { return symbol(sym.MODOP); }
"OR"        { return symbol(sym.OR); }
```

Listing 6: oberon.flex — 整数与标识符规则

```
{DecimalInteger} {
    // Check len > 12 → IllegalIntegerRangeException
```

```
// Check next is letter → IllegalArgumentException
return symbol(sym.INTEGER_LITERAL, Integer.parseInt(yytext()));
}

{OctalInteger} {
    // Check len > 12 → IllegalIntegerRangeException
    // Check contains '8'/'9' → IllegalOctalException
    return symbol(sym.INTEGER_LITERAL, Integer.parseInt(yytext(), 8));
}

{Identifier} {
    // Check len > 24 → IllegalIdentifierLengthException
    return symbol(sym.IDENTIFIER, yytext().toLowerCase());
}
```

Listing 7: oberon.flex — 注释与错误恢复

```
"(*) {CommentContent}* "*" + ")" { /* ignore comment */ }
"(*) {CommentContent}* {
    throw new MismatchedCommentException(...);
}

// Fallback: any unrecognized character
[^] {
    throw new IllegalSymbolException(...);
}
```

6 异常类层次设计

词法分析阶段的异常全部继承自 `LexicalException`，最终汇聚到 `OberonException` 根类：



图 6: 词法异常类层次——6 个具体异常 + 触发条件

每条异常类都是独立的.java 文件，遵循简单、可抛的原则：

Listing 8: IllegalOctalException.java — 典型词法异常类

```
package exceptions;

public class IllegalOctalException extends LexicalException {
    public IllegalOctalException(String message) {
        super(message);
    }
}
```

7 Lex 族工具差异比较

7.1 三代 Lex 工具对比

特性	JFlex (2000s)	JLex (1990s)	GNU Flex (1980s)
实现语言	Java	Java	C
生成目标	Java	Java	C
词法引擎	优化的 DFA 表	基础 DFA	优化的 DFA 表
正则语法	Unicode、预定义类	基本 POSIX	POSIX 字符类
%cup 集成	内置%cup 指令	需手动配置	需手动配合 yacc
输入格式	.flex	.lex	.l
维护状态	活跃维护 (1.8.2, 2020)	停止维护	活跃维护
性能	中等 (JVM 优化可补偿)	较慢	最快 (原生 C)
状态处理	%state 词法状态	%state	Start conditions
用户文档	完整手册 + 示例	基础文档	完整 texinfo 手册

表 2: JFlex / JLex / GNU Flex 十维对比

7.2 关键差异分析

1. **语言生态**: 选择哪种工具首先取决于目标语言。JFlex → Java 生态 (配合 JavaCUP), GNU Flex → C/C++ 生态 (配合 Bison)。JLex 是 JFlex 的前身, 已停止维护
2. **DFA 优化**: JFlex 生成的 DFA 表比 JLex 更紧凑——它使用更优的 NFA→DFA 转换算法, 生成的表大小约为 JLex 的 60–80%
3. **Unicode 支持**: JFlex 天然支持 Unicode (`\p{Alpha}` 等), 这对处理 Oberon-0 大小写无关的标识符特别有用。GNU Flex 的 Unicode 支持需要额外配置
4. **CUP 集成**: JFlex 的 %cup 指令自动生成 `java_cup.runtime.Symbol` 返回值和与新式的 JavaCUP 完全适配的接口, 而 JLex 需要手工修改生成的代码
5. **性能对比**: GNU Flex 生成的原生 C 代码理论最快, 但在 Oberon-0 这种小语言的测试中差异不显著

8 测试结果

8.1 正确程序扫描

Listing 9: Sample.obr 扫描输出 (前 10 行)

```
Token{MODULE, module} @15:2
Token{IDENTIFIER, sample} @15:9
```

```

Token{SEMICOLON, ;} @15:15
Token{CONST, const} @16:4
Token{IDENTIFIER, maxaccounts} @17:6
Token{EQ, =} @17:18
Token{INTEGER_LITERAL, 100} @17:20
Token{SEMICOLON, ;} @17:23
Token{IDENTIFIER, bonusrate} @18:6
Token{EQ, =} @18:16

```

8.2 词法错误检测汇总

用例	输入特征	抛出异常	结果
Sample.obr	完整正确源程序	— (无异常)	PASS
Sample.001	5 @ 3	IllegalSymbolEx.	PASS
Sample.002	25id 无空白	IllegalIntegerEx.	PASS
Sample.003	标识符 > 24 字符	IllegalIdLenEx.	PASS
Sample.004	(* unclosed	MismatchedCommEx.	PASS
Sample.005	018 八进制非法	IllegalOctalEx.	PASS
Sample.006	整数 > 12 位	IllegalIntRangeEx.	PASS

表 3: 词法测试结果: 6/6 错误检测通过 (异常名缩写: Ex.=Exception, IdLen=IdentifierLength, Comm=Comment, IntRange=IntegerRange)

实验二完成: 词汇表 (19 保留字 + 7 关键字含分类理由)、6 条正则定义式 + 6 条约束规则、oberon.flex (JFlex 输入源文件完整三段结构)、OberonScanner.java (含整数 DFA / 标识符识别 / 注释扫描 / 双字符运算符处理)、16 个异常类的层次设计 (3 层继承)、JFlex/JLex/GNU Flex 十维差异比较。词法错误检测 6/6 通过, 正确程序 token 化完整正确。